

Maximum Likelihood Program

Maximum Likelihood Estimation (MLE) sounds grandiose, but the idea is simple: adjust your model so that the text you actually observed becomes as probable as possible. In language modeling that means increasing the probabilities of the token transitions that appear in the training data. A small program can demonstrate this using bigram counts—counts of word pairs—which are then converted into probabilities. The counting step is essentially performing MLE for this simple model

Step 1 — Import Required Library

This provides convenient data structures for counting word occurrences.

```
from collections import defaultdict, Counter
```

Step 2 — Training Corpus

This small dataset represents the observed text that the model should make highly probable

```
corpus = [  
    "the cat sat on the mat",  
    "the cat ate the fish",  
    "the dog sat on the rug",  
    "the dog ate the bone"  
]
```

Step 3 — Tokenization Function

This converts each sentence into a list of words.

```
def tokenize(sentence):  
    return sentence.lower().split()
```

Step 4 — Build Bigram Counts (Learning From Data)

```
bigram_counts = defaultdict(Counter)
```

```
for sentence in corpus:
```

```
    tokens = tokenize(sentence)
```

```
    for i in range(len(tokens)-1):
```

```
        w1 = tokens[i]
```

```
        w2 = tokens[i+1]
```

```
        bigram_counts[w1][w2] += 1
```

Example counts after training might look like:

the → {cat:2, dog:2, mat:1, fish:1, rug:1, bone:1}

cat → {sat:1, ate:1}

dog → {sat:1, ate:1}

The model has now learned from the observed data.

Step 5 — Convert Counts to Probabilities (MLE)

MLE says:

$$P(w_t | w_{t-1}) = \text{count}(w_{t-1}, w_t) / \text{count}(w_{t-1})$$

The program computes these probabilities

```
bigram_probs = {}
```

```
for word, counter in bigram_counts.items():
```

```
    total = sum(counter.values())
```

```
    bigram_probs[word] = {
```

```
        next_word: count / total
```

```
        for next_word, count in counter.items()
```

```
    }
```

Now the model assigns higher probability to transitions that appeared more often.

Step 6 — Function to Compute Sentence Probability

This applies the language modeling objective

```
def sentence_probability(sentence):  
    tokens = tokenize(sentence)  
  
    probability = 1.0  
  
    for i in range(len(tokens)-1):  
  
        w1 = tokens[i]  
  
        w2 = tokens[i+1]  
  
        if w1 in bigram_probs and w2 in bigram_probs[w1]:  
  
            probability *= bigram_probs[w1][w2]  
  
        else:  
  
            probability *= 0.00001  
  
    return probability
```

Step 7 — Test the Model

```
s1 = "the cat sat on the mat"  
  
s2 = "the dog ate the bone"  
  
print("Sentence 1 probability:", sentence_probability(s1))  
  
print("Sentence 2 probability:", sentence_probability(s2))
```

Because these sentences appear in the training corpus, the model assigns them relatively high probabilities.

Step 8 — Test an Unlikely Sentence

```
s3 = "the fish sat on the dog"
```

```
print("Sentence 3 probability:", sentence_probability(s3))
```

The probability will be very small because the sequence was not observed in training data.